漫步数码时代，

铸就精彩人生，

程序员修炼之路，

从精雕算法开始……

第5章 回溯法

(Back Tracking)

(一)

主讲人：周正深

5.1 回溯算法的基本思想和适用条件

5.2 回溯算法的设计步骤

5.3 回溯算法的效率估计和改进途径

5.4 分支限界

5.4.1 背包问题

5.4.2 最大团问题

5.4.3 货郎问题

5.4.4 圆排列问题

5.4.5 连续邮资问题

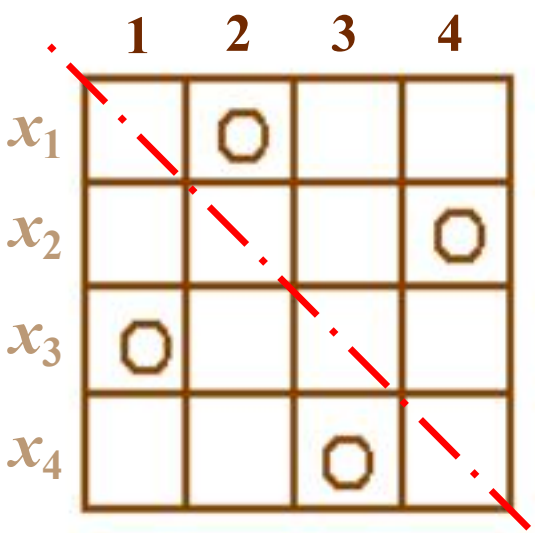
5.1.1 几个典型例子

1. 四后问题
2. 0-1背包问题
3. 货郎问题（*TSP*）

5.1.2 回溯算法的适用条件

例5.1 *n*后问题

4后问题：在4×4方格的棋盘上放置4个皇后，使得没有2个皇后在同一行、同一列、也不在同一条45度的斜线上，问有多少种可能的布局。



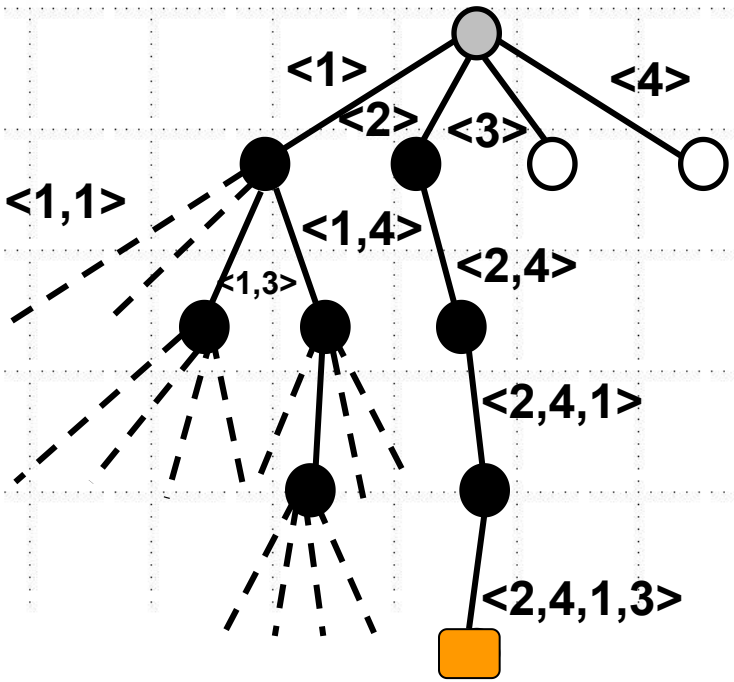
解是一个4维向量(列号)，

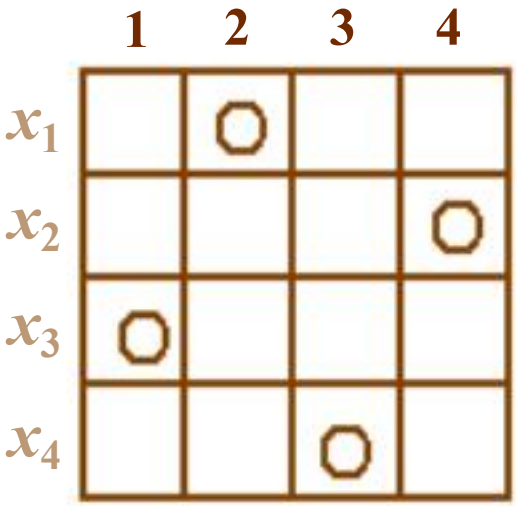
$$<x_1, x_2, x_3, x_4>$$

解：< 2，4，1，3 >,< 3，1，4，2 >

两个解成对角线对称

搜索空间 4叉树





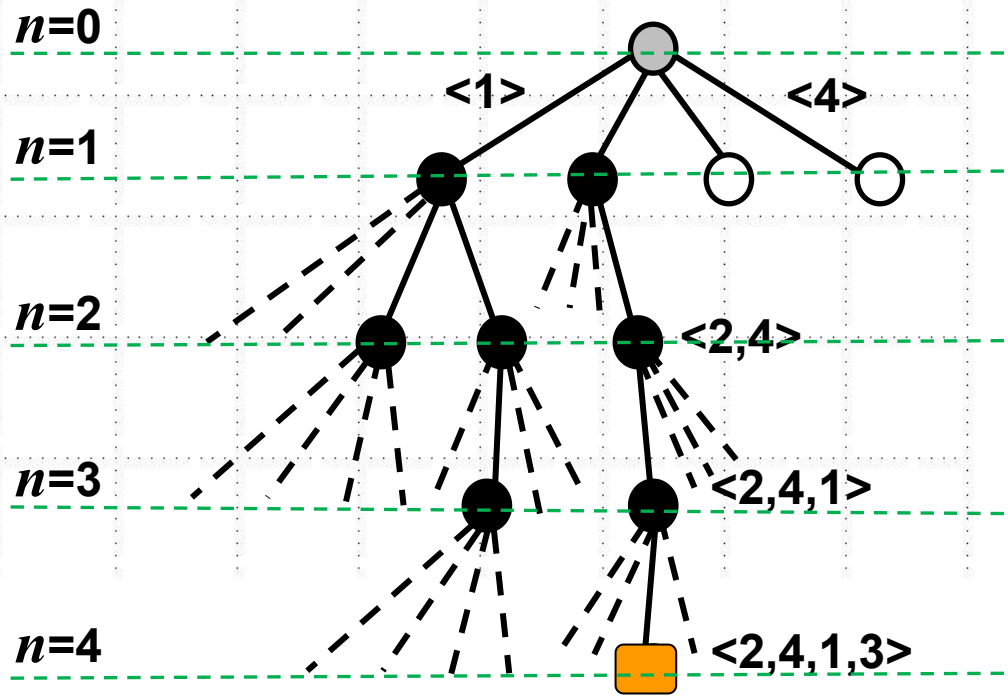
每个结点有4个儿子，分别代表选择1,2,3,4列的位置

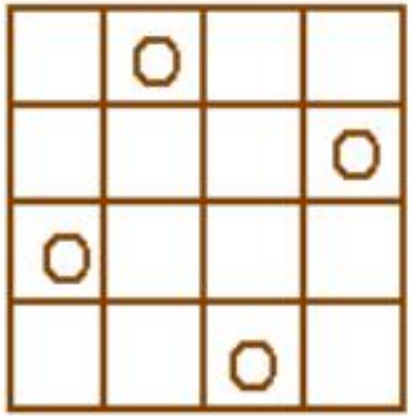
第i层选择解向量中第i个分量的值，

最深层的树叶是解

按深度优先的次序遍历树，找到所有解。

搜索空间 4叉树，4^4片树叶





由于有4^4片树叶，搜索空间n^n，回溯法算法最坏时间复杂度可降为$O(n!)$

跳跃式遍历 通过约束条件可以剪枝，适当降低复杂度。

8后问题

解是一个8维向量，$<x_1, x_2, x_3, x_4, x_5, x_6, x_7, x_8>$

搜索空间：8叉树，8^8片树叶

有92个解。

例如：<1, 5, 8, 6, 3, 7, 2, 4>是一个解

由于有8^8片树叶，所以搜索空间为n^n，回溯法可以降为算法时间复杂度$O(n!)$。

n后问题

解是一个n维向量，$<x_1, x_2, \ldots, x_n>$

搜索空间：n叉树，n^n片树叶

由于有n^n片树叶，回溯法算法时间复杂度$O(n!)$。

5.1.1 几个典型例子

8皇后问题的一个解：

<1, 5, 8, 6, 3, 7, 2, 4>

9后问题：

352个解，运行10s

10后问题：

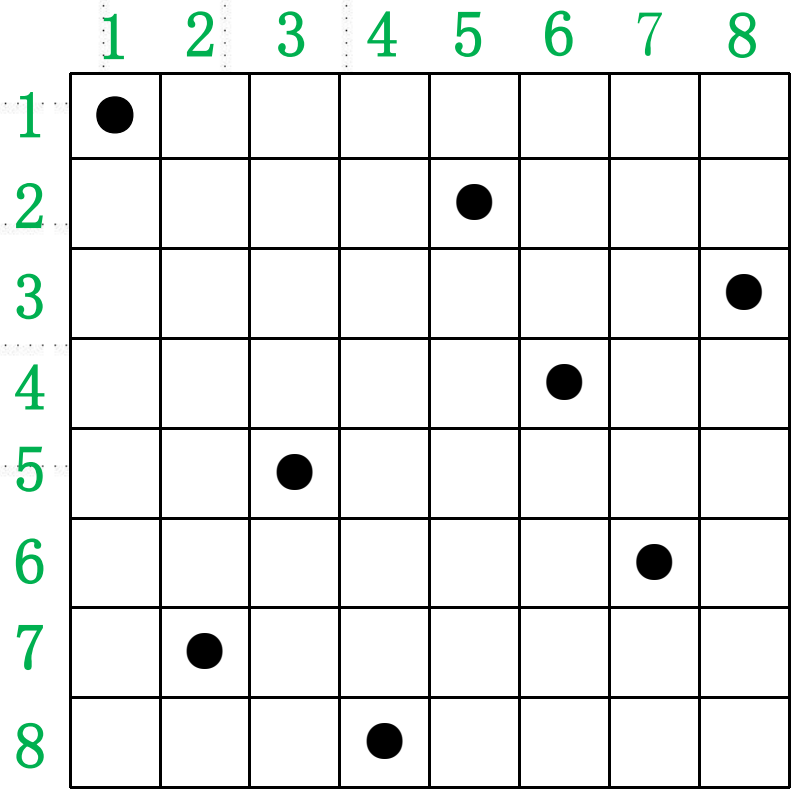
724个解，运行25s

11后问题：

2680个解，运行95s

12后问题：

14200个解，运行511s



八皇后问题

例5.2 0-1背包问题

有n种物品，每种物品只有1个，第i种物品价值为v_i，重量为w_i，i=1,2,…,n。问如何选择放入背包的物品，使得总重量不超过B，而价值达到最大。

实例：

V={12,11,9,8}, W={8,6,4,3}, B=13

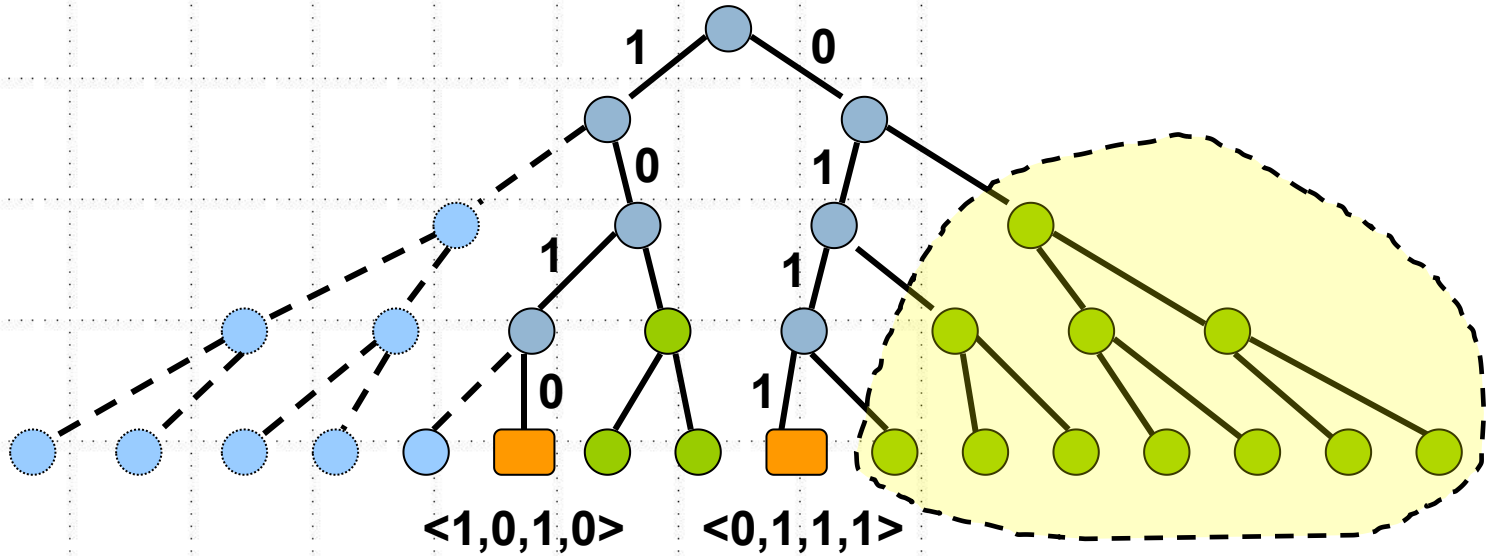
最优解：

<0, 1, 1, 1>，价值：28，重量13

例5.2　0-1背包问题

实例：V={12,11,9,8}, W={8,6,4,3}, B=13

搜索空间　子集树



<1,0,1,0> 可行解：　x_1=1, x_2=0, x_3=1, x_4=0. 价值:21，重量:12

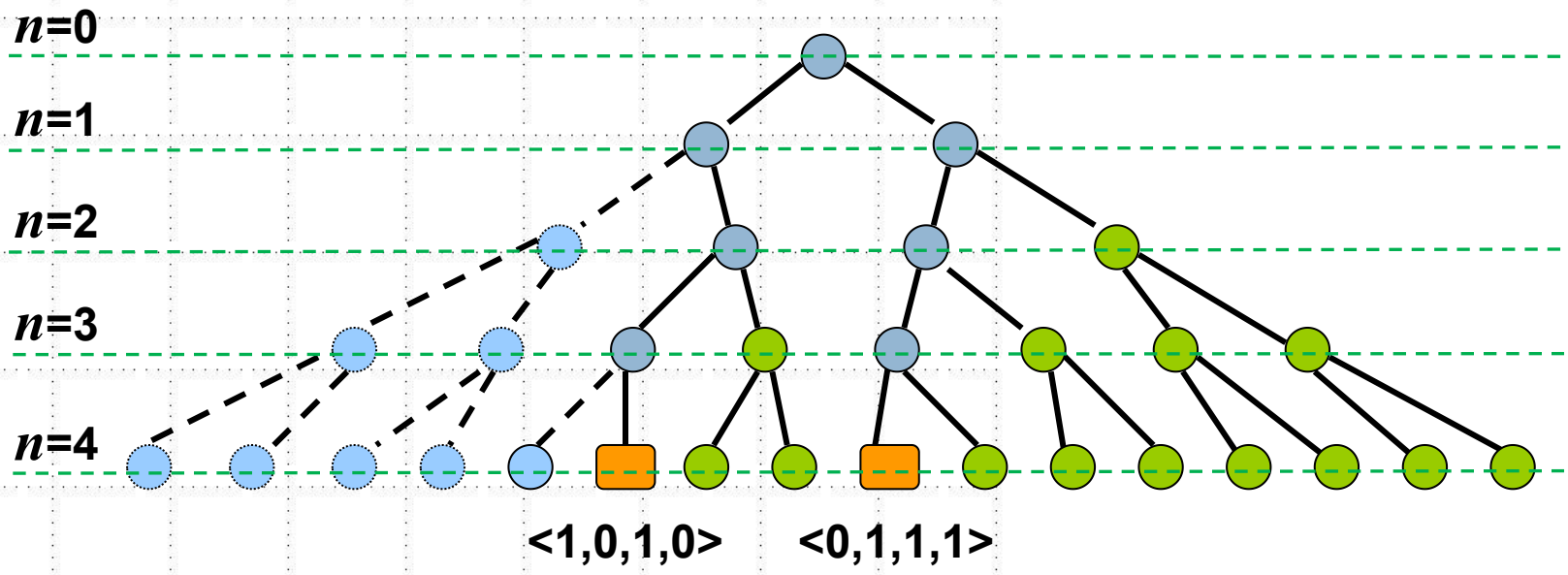
<0,1,1,1> 可行解：　x_1=0, x_2=1, x_3=1, x_4=1. 价值:28，重量:13

……

比较完所有可行解，得知价值最大的最优解为：<0,1,1,1>

例5.2　0-1背包问题

搜索空间　子集树，2^n片树叶



由于有2^n片树叶结点需要搜索，所以算法时间复杂度$O(2^n)$

跳跃式遍历　通过约束条件，可以修剪一些树枝，可适当简化算法复杂度。

例5.3 货郎问题

有n个城市，以及任意两个城市之间的距离，求一条每个城市恰好经过一次的回路，使得总长度最小。

建模： 城市集 $C=\{c_1, c_2, \ldots, c_n\}$,

距离 $d(c_i, c_j)=d(c_j, c_i) \in Z^+$，$1 \le i < j \le n$

求：$1, 2, \ldots, n$的排列$k_1, k_2, \ldots, k_n$使得

$$\min\{\sum_{i=1}^{n-1} d(c_{k_i}, c_{k_{i+1}}) + d(c_{k_n}, c_{k_1})\}$$

例5.3　货郎问题

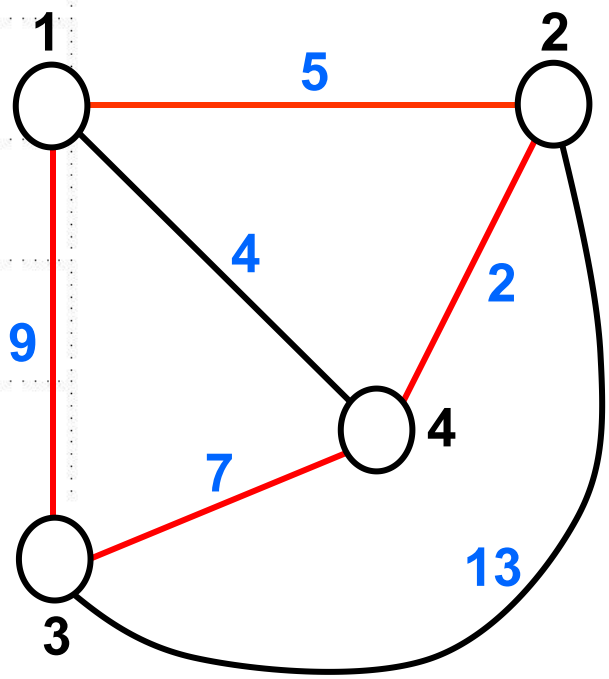
实例

$C=\{1, 2, 3, 4\}$

$d(1, 2)=5$，　$d(1, 3)=9$，

$d(1, 4)=4$，　$d(2, 3)=13$，

$d(2, 4)=2$，　$d(3, 4)=7$，

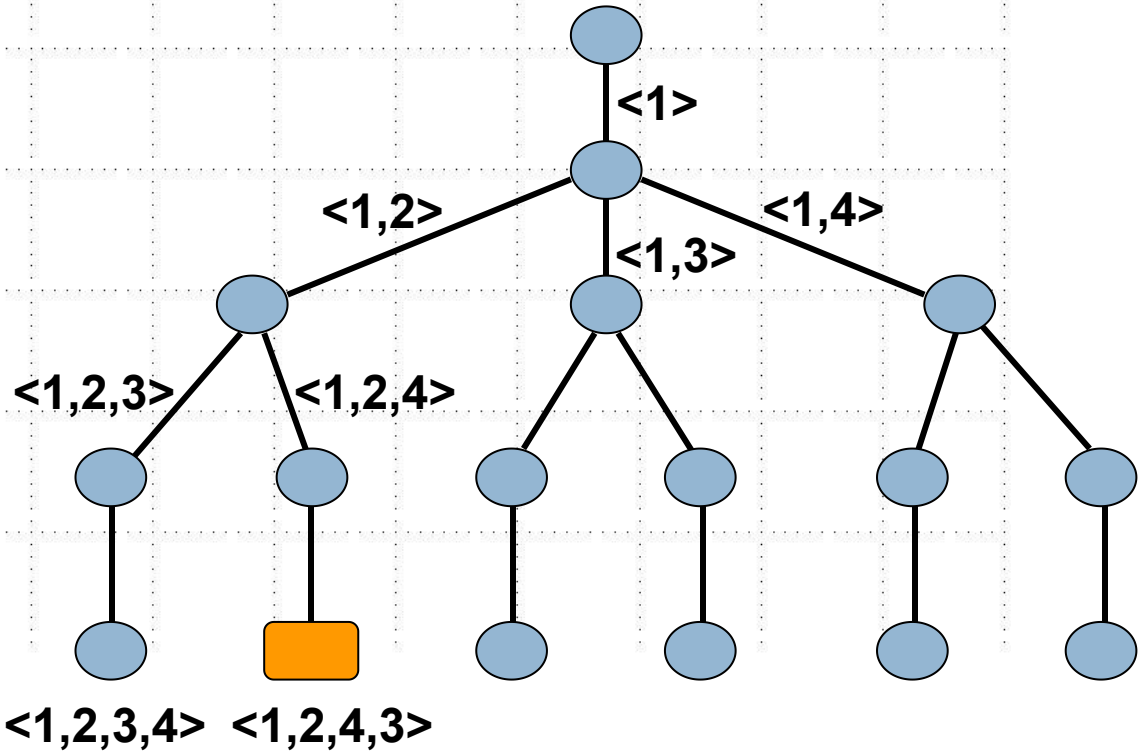


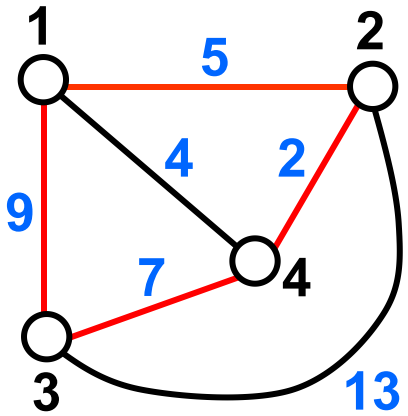
解：$<1, 2, 4, 3, 1>$，

长度 $= 5+2+7+9=23$

例5.3 货郎问题

搜索空间 排列树





<1,2,3,4>对应于巡回路线:1→2 →3 →4 →1 长度:5+13+7+4=29

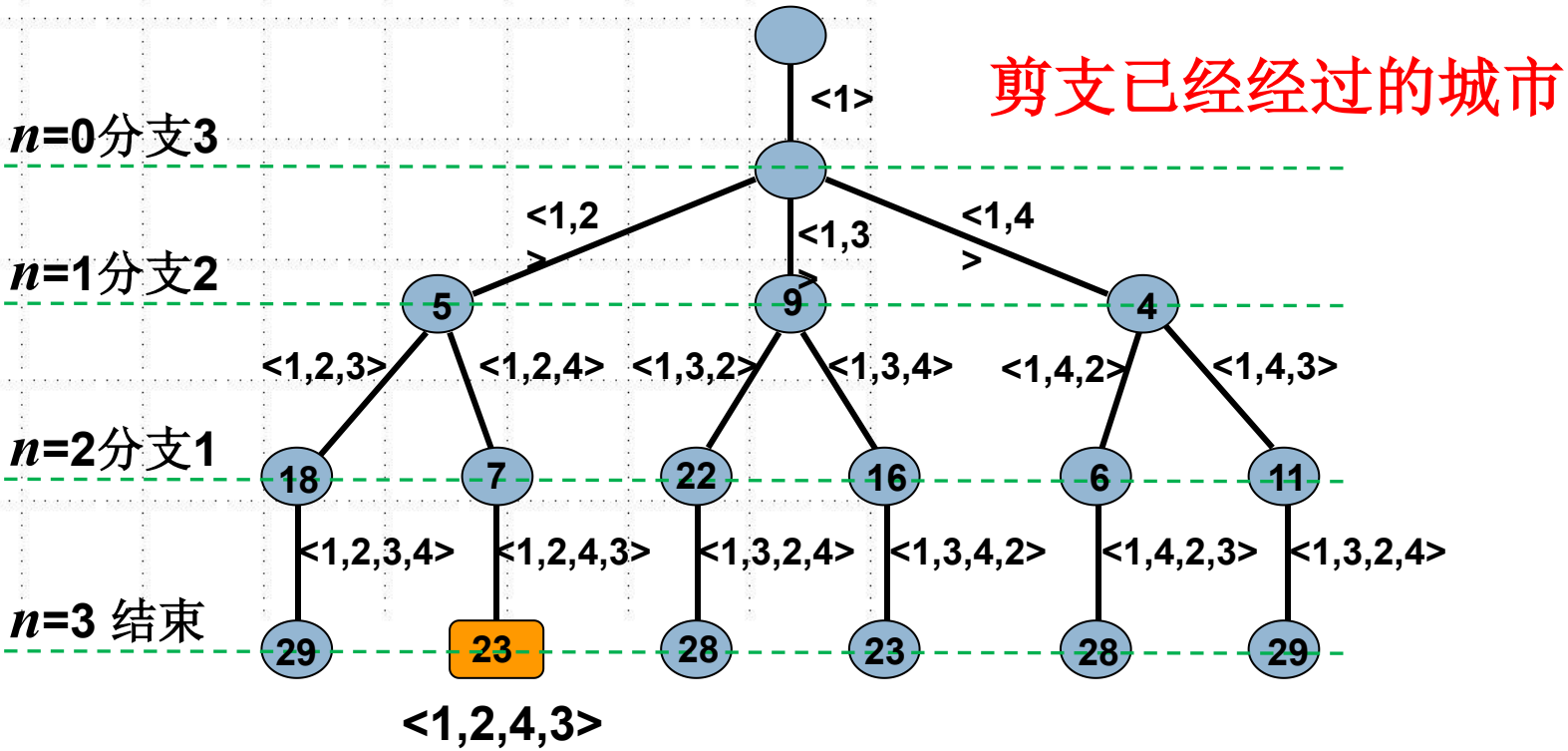
<1,2,4,3>对应于巡回路线:1→2 →4 →3 →1 长度:5+2+7+9=23

……

最优解：<1,2,4,3>，长度：23

例5.3 货郎问题

搜索空间 排列树，本身有$(n-1)^{n-1}$片树叶，剪支后$(n-1)!$片。



由于有$(n-1)!$片树叶，算法时间复杂度$O((n-1)!)$

跳跃式遍历 利用前边搜索的可能解的最小值剪枝，适当降低算法的复杂性。

5.1.1 回溯算法例子小结

问题分析

不能宽度优先

问题	解性质	解描述向量	搜索空间	搜索方式	约束条件
n后	可行解	$<x_1, x_2, x_3, x_4>$ x_i：第i行列号	n叉树 $O(n^n)$	深度、 函数优先	彼此 不攻击
0-1背包	最优解	$<x_1, x_2, x_3, x_4>$ x_i=0,1 选/不选	子集树 $O(2^n)$	深度、 函数优先	不超 背包重量
货郎	最优解	$<k_1, k_2, \ldots, k_n>$ 1,2,…,n的排列	排列树 $O((n-1)^{n-1})$	深度、 函数优先	选没有经 过的城市
特点	搜索解	向量：不断扩 张部分向量	树 ≥指数级	跳跃式 遍历	约束条件 回溯判定

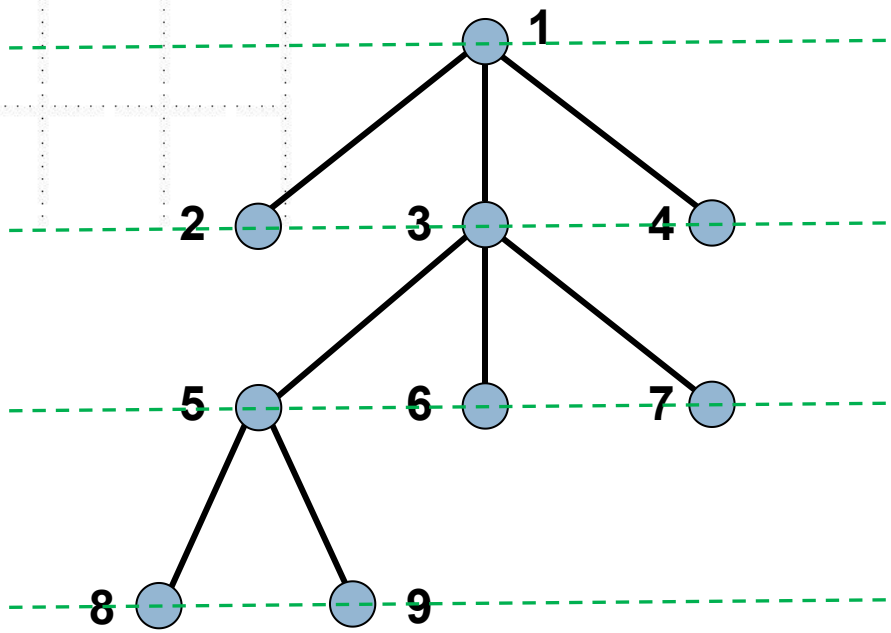
回溯法与深度优先搜索的关系

深度优先搜索　深度优先搜索思想是按一个分支“一直向下走，走到底再掉头”，回溯法是“一直向下走，走不通就掉头”，深度优先搜索是回溯法的一种特例。区别是回溯法在求解过程中不必访问全部结点，而深度优先搜索则访问完整的搜索树。

深度优先访问次序：

1 → 2 → 3 → 5 →

8 → 9 → 6 → 7 → 4



5.1.1 回溯算法的基本思想

(1)适用问题：求解搜索问题和优化问题

(2)搜索空间：树，结点对应部分解向量，树叶对应可行解

(3)搜索过程：采用系统的方法隐含遍历搜索树

(4)搜索策略：深度优先，函数优先，或互相结合等

(5)结点分支判定条件：

满足约束条件 → 分支扩张解向量

不满足约束条件 → 回溯到该结点的父结点

(6)结点状态：动态生成

白结点(尚未访问)；

灰结点(正在访问以该结点为根的子树)；

黑结点(以该结点为根的子树遍历完成)

(7)存储：当前路径

结点状态

深度优先访问次序： 1 → 2 → 3 → 5 → 8

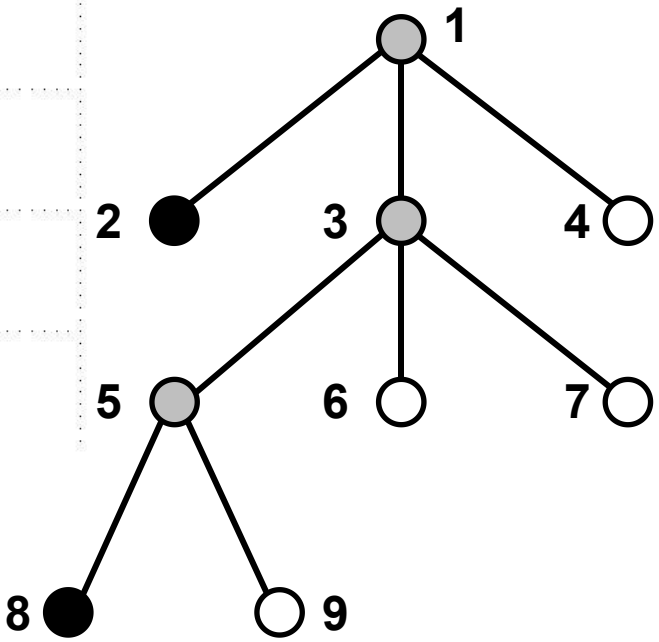
如图表示：当前所处的访问状态

①已完成访问：2，8

②已访问但未结束：1，3，5

③未访问：4，6，7，9

④下一个该访问的结点是：？



回溯算法跳跃式遍历的适用条件

设 $P(x_1, x_2, \ldots, x_i)$ 为真表示向量$<x_1, x_2, \ldots, x_i>$ 满足某个性质 (n后问题中i 个皇后放置在彼此不能攻击的位置）

多米诺性质：

$$P(x_1, x_2, \ldots, x_{k+1}) \rightarrow P(x_1, x_2, \ldots, x_k) \qquad 0 < k < n$$

逆否命题：

$$\neg P(x_1, x_2, \ldots, x_k) \rightarrow \neg P(x_1, x_2, \ldots, x_{k+1}) \qquad 0 < k < n$$

k维向量不满足约束条件，扩张向量到k+1维仍旧不满足，可以回溯。

5.1.2 一个反例

例5.4 求不等式的整数解

$$5x_1+2x_2-x_3 \le 10, \quad 1 \le x_i \le 3,\ i=1,2,3$$

$P(x_1, \ldots, x_k)$：意味将$x_1, x_2, \ldots, x_k$代入原不等式的相应部分使得左边小于等于10，不满足多米诺性质.

通过变换使得该问题满足多米诺性质.

变换： 令 $x_3=3-x_3'$,

$$5x_1+2x_2+x_3' \le 13\ ,\ 1 \le x_1, x_2 \le 3, 0 \le x_3' \le 2$$

5.1.2 一个反例

解： (1)设计思想：采用回溯法，按x_1，x_2，x_3，深度优先搜索不等式的整数解。

(2)算法伪码　$5x_1+2x_2+x_3' \le 13$，$1 \le x_1, x_2 \le 3, 0 \le x_3' \le 2$

```
for x1=1 to 3 do
    if 5x1 > 13 then break //剪枝条件
    for x2=1 to 3 do
        if 5x1+2x2 > 13 then break //剪枝条件
        for x3' =0 to 2 do
            if 5x1+2x2+x3' > 13 then break //剪枝回溯
            else 输出x1，x2，x3 //输出解
```

回溯算法小结

一、回溯算法的适用条件

多米诺性质

二、回溯算法的设计步骤

(1) 定义搜索问题的解向量和每个分量的取值范围

确定解向量 $<x_1, x_2, \ldots, x_n>$

确定 x_i 的可能取值的集合为 X_i , $i = 1, 2, \ldots, n$.

(2) 确定结点儿子的排列规则

(3) 当 $x_1, x_2, \ldots, x_{k-1}$ 确定以后，计算 x_k 取值集合S_k, $S_k \subseteq X_k$

(4) 判断是否满足多米诺性质

(5) 搜索策略——深度优先、函数优先等

(6) 确定每个结点分支约束条件

作业

习题5 P132 5.1、5.2(上机编程)

5.1 用回溯法求解下列不等式的所有的整数解。给出伪码和解。

$$\begin{cases} 3x_1+4x_2+2x_3 <= 12 \\ x_1, x_2, x_3 \text{ 为非负整数} \end{cases}$$

作业

5.2 最小重量机器设计问题。某设备需要四种配件，每种一件。有三个供应商提供这些配件。下表给出有关的价格和每种配件的重量。从中选择这四种配件，使得总价值不超过120，总重量最轻。

产品和供应商信息表

零件编号	供应商1		供应商2		供应商3	
	价格	重量	价格	重量	价格	重量
1	10	5	8	6	12	4
2	20	8	21	10	30	5
3	40	5	42	4	30	10
4	30	20	60	10	45	15

结 束

谢谢收看，再见！